

Backup and recovery issues

dr hab. inż. Maciej Grzenda, prof. uczelni

M.Grzenda@mini.pw.edu.pl

<http://www.mini.pw.edu.pl/~grzendam>

Politechnika Warszawska
Wydział Matematyki i Nauk Informacyjnych



**European
Funds**

Knowledge Education Development

**Warsaw University
of Technology**

European Union
European Social Fund



MSc program in Data Science has been developed
as a part of task 10 of the project
„NERW PW. Science - Education - Development - Cooperation”
co-funded by European Union from European Social Fund.

Introduction

- To start RDBMS instance successfully, we need to ensure that all the mandatory files are present and valid
- This means we need:
 - data files
 - transaction log files (called redo log files in Oracle)
 - but also configuration files such as spfile and control files
- Hence, a DBA has to know how to:
 - Start DBMS instance also in the case of troubles
 - Backup necessary files and recover the instance after media failure and other problems

References

- <https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlug/STARTUP.html>
- <https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlug/SHUTDOWN.html>
- <https://docs.oracle.com/en/database/oracle/oracle-database/19/bradv/index.html>

Starting up Oracle instance

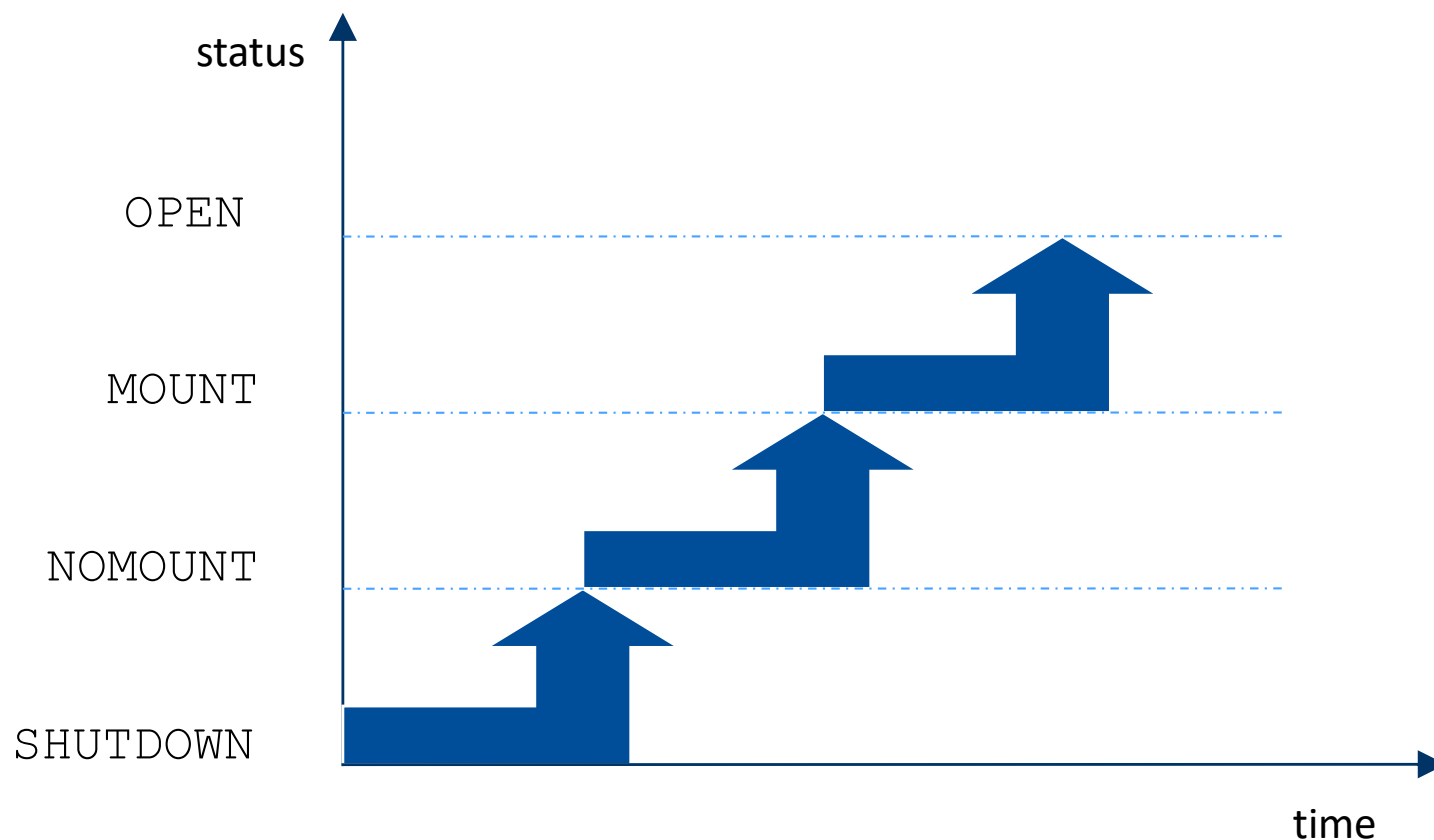
Introduction

- Oracle instance can be in many states, not just running or not running
- These states are used to gradually try to open individual file categories
- In case, extra recovery works are needed, a DBA has to be able to start a database gradually to identify and resolve potential problems such as a corrupt data file, which may happen due to disk failure

Database states

State	Description	Files needed to enter the state
SHUTDOWN	All files are closed. The instance does not exist i.e. there are no Oracle processes/memory areas.	
NOMOUNT	The instance is started. SGA is established. Background processes are running. However, the instance is not connected to a database. Can be used to create database or recreate control files.	Initialisation parameter files
MOUNT	In addition to NOMOUNT, all control files listed in initialisation files are checked. In case, at least one of them does not exist or is damaged, the instance remains in NOMOUNT state.	All database control files have to exist and be identical
OPEN (the status in which database is fully accessible)	In addition to MOUNT, the instance checks whether: •at least one redo log file in each group is present and correct, •all on-line data files are present and correct, •all data files that are not on-line and not read-only are synchronised with control file	At least one redo log file in each group All on-line data files Location of online logs and data files specified by control files

Opening a database



Before a database is fully opened, it needs to successfully pass start-up stages. Starting from SHUTDOWN, it goes through NOMOUNT and MOUNT to finally reach OPEN stage, which means that it is fully available and fully synchronised at the same time.

Changing instance status

- Startup:
 - In sqlplus, when instance is shut down:
 - `STARTUP [NOMOUNT | MOUNT | OPEN | RESTRICT]`
 - When in NOMOUNT state:
 - `ALTER DATABASE MOUNT | OPEN`
- Shutdown:
 - When in NOMOUNT, MOUNT or OPEN:
 - `SHUTDOWN NORMAL | TRANSACTIONAL | IMMEDIATE | ABORT`

STARTUP with no arguments tries to mount and open the database. RESTRICT option serves to mount and open database, but let only users with RESTRICTED SESSION privilege can connect. It is used for administrative needs. Please note that selected options only are shown here.

SHUTDOWN settings

Option	Meaning
NORMAL	No new connections are permitted. Default setting. The instance shuts down, but only after all users decide to log off. Rarely used, as typically at least Oracle processes are connected.
TRANSACTIONAL	No new connections are permitted. The instance shuts down, but only after all sessions finish their transactions. Sessions not in a transaction are terminated immediately.
IMMEDIATE	No new connections are permitted. The instance shuts down. Active transactions are rolled back. All sessions are terminated then.
ABORT	The instance terminates immediately – equivalent to power failure. No database files are synchronised. Uncommitted transactions are not rolled back. Still, basing on redo log files an instance recovery will be performed next time the instance is opened. Should be used when other shutdown types do not work.

Normal shutdown modes

- A term used to refer to NORMAL, TRANSACTIONAL and IMMEDIATE
- When shutting down the database in one of these modes:
 - PMON process rolls back any incomplete transactions,
 - CKPT process executes a checkpoint which forces the DBWn process to write all the data from buffer cache to the data files,
 - File headers are updated and files closed,
 - The database is ready to be used next time it will be opened – no uncommitted transactions in progress, all data files and log files synchronised.

For the reasons listed above, the ABORT mode is referred to as disorderly shutdown meaning that the database may contain uncommitted changes and may miss committed changes. In other words, quite likely at first restart instance recovery will be needed. Still, even when ABORT mode is used there is no risk of data loss.

Data storage and file types in Oracle DBMS

References and further reading

1. Physical storage:
<https://docs.oracle.com/en/database/oracle/oracle-database/19/cncpt/physical-storage-structures.html>
2. Logical storage:
<https://docs.oracle.com/en/database/oracle/oracle-database/19/cncpt/logical-storage-structures.html>
3. Tablespaces:
<https://docs.oracle.com/en/database/oracle/oracle-database/19/admin/managing-tablespaces.html>

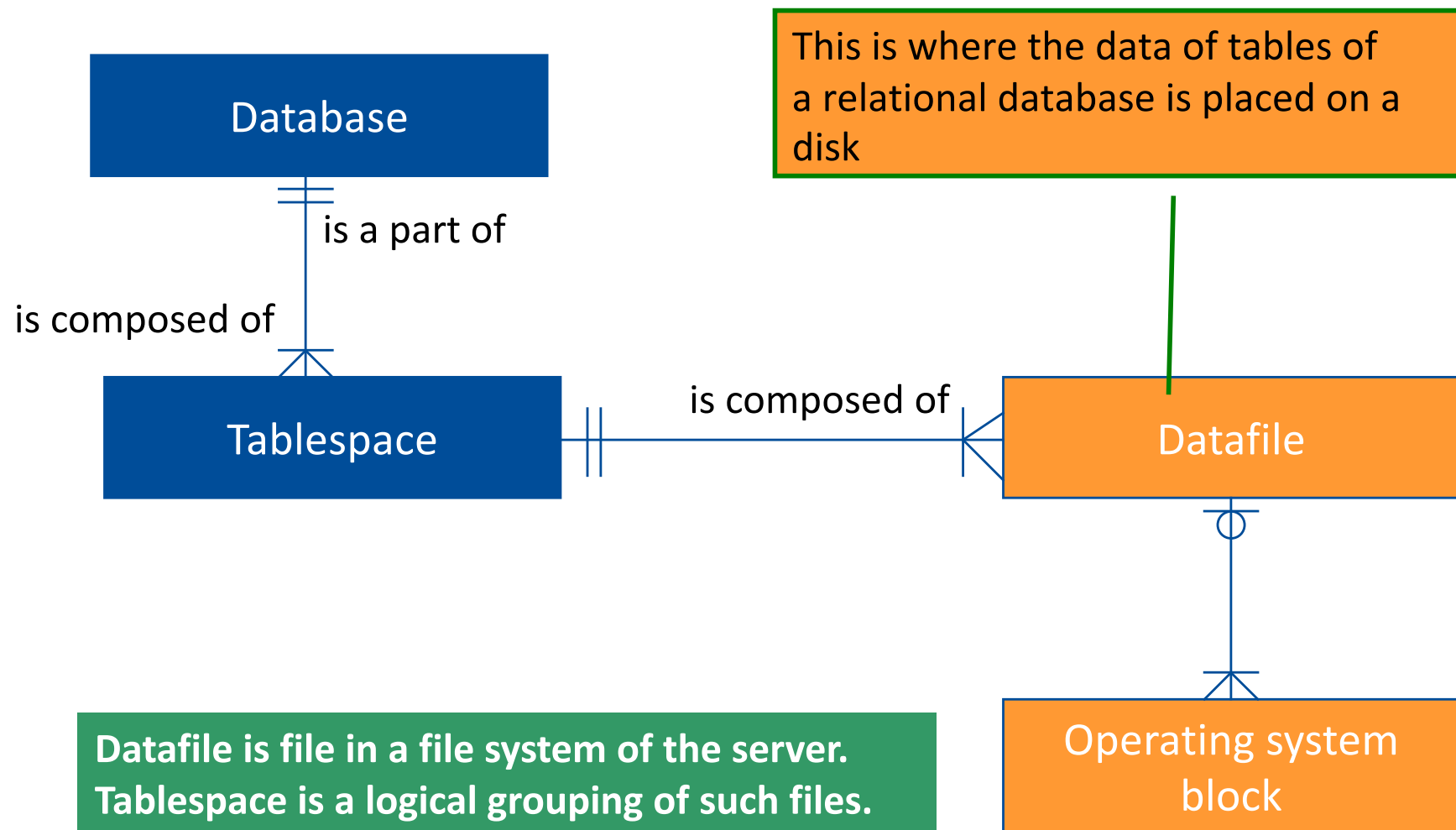
Key solutions

- Logical structures such as tables, views and indexes are independent from physical structures such as files
- In particular:
 - the data of many tables may reside in one file
 - the data of one table may reside in many files
- Hence, both logical data structures (such as tables) and physical data storage settings (such as data files) are managed largely independently

Oracle storage

- Two aspects:
 - *Physical storage* refers to the way operating system sees the Oracle files containing the data of a database
 - *Logical storage* refers to the way Oracle internally manages the data within data files
- Both aspects have to be addressed by DBA to properly allocate disk space, handle backups and ensure required performance

Physical storage



Tablespaces

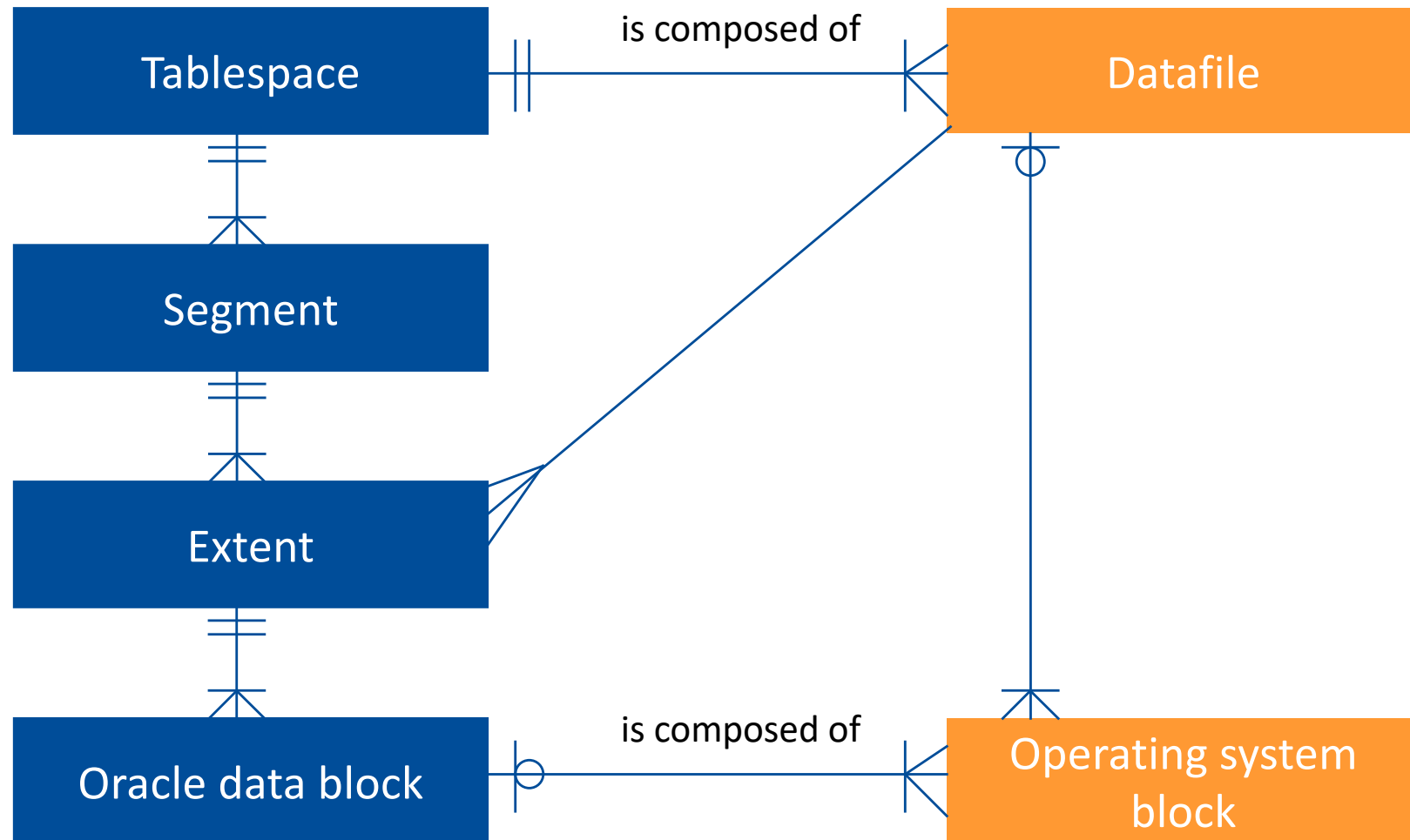
Przestrzenie tabel

Tablespaces

Przestrzenie tabel

- A tablespace is a group of data files
- For every tablespace, its own unique backup strategy can be deployed
- A tablespace is assigned to every table to make it clear where the data of this table will be allocated
- Multiple tablespaces are used by Oracle database in order to:
 - Separate system data such as metadata from user data i.e. the data of user tables. This is done *inter alia* to reduce I/O contention
 - Enable different backup policies for different subsets of data, e.g. more frequent backups for some of the data
 - Make it possible to have different data in different files and possibly make them use different storage options such as OS file system vs. ASM. It is also possible to simply place the data of different tablespaces on different disk drives for increased performance
- Hence, planning tablespaces is one of the key tasks of a DBA

Logical storage vs. physical storage



Legend:

Logical storage

Physical storage

Tablespace categories

- Each tablespace can store just one category of segments:
 - *permanent* – keeping persistent schema objects e.g. table or index data. Such objects are stored in data files.
 - *temporary* – contains objects existing only temporarily; it is used e.g. to keep the result of sorting or the data of a temporary table. Objects in such tablespace are kept in temporary files.
 - *undo* – hold the image of the data before modifications so that queries of other users can be answered in spite of the fact that the data is being modified at the moment. There can be many undo tablespaces per a database, but only one of them will be active.

When you create a new user in Oracle, you define default permanent tablespace (for keeping the data of the user) and temporary tablespace used to store temporary segments for the sessions of this user. There should be at least one temporary tablespace in each database.

Tablespace attributes

A tablespace has some attributes. For instance it can be made read only to prevent changes in this tablespace

Database Instance: orcl > Tablespaces > Create Tablespace

Create Tablespace

General | **Storage**

* Name

Extent Management	Type	Status
☒ Locally Managed	☒ Permanent	☒ Read Write
☐ Dictionary Managed	☐ Set as default permanent tablespace	☐ Read Only
	☐ Temporary	☐ Offline
	☐ Set as default temporary tablespace	
	☐ Undo	
	Undo Retention Guarantee ☐ Yes ☒ No	

All the datafiles of a tablespace share the same database block size, typically 4KB or 8KB. It should be a multiple of the operating system block size. Whenever data is read, entire database blocks are read. Similarly whenever the data is changed, entire database blocks are saved. You can not change the database block size for an existing tablespace.

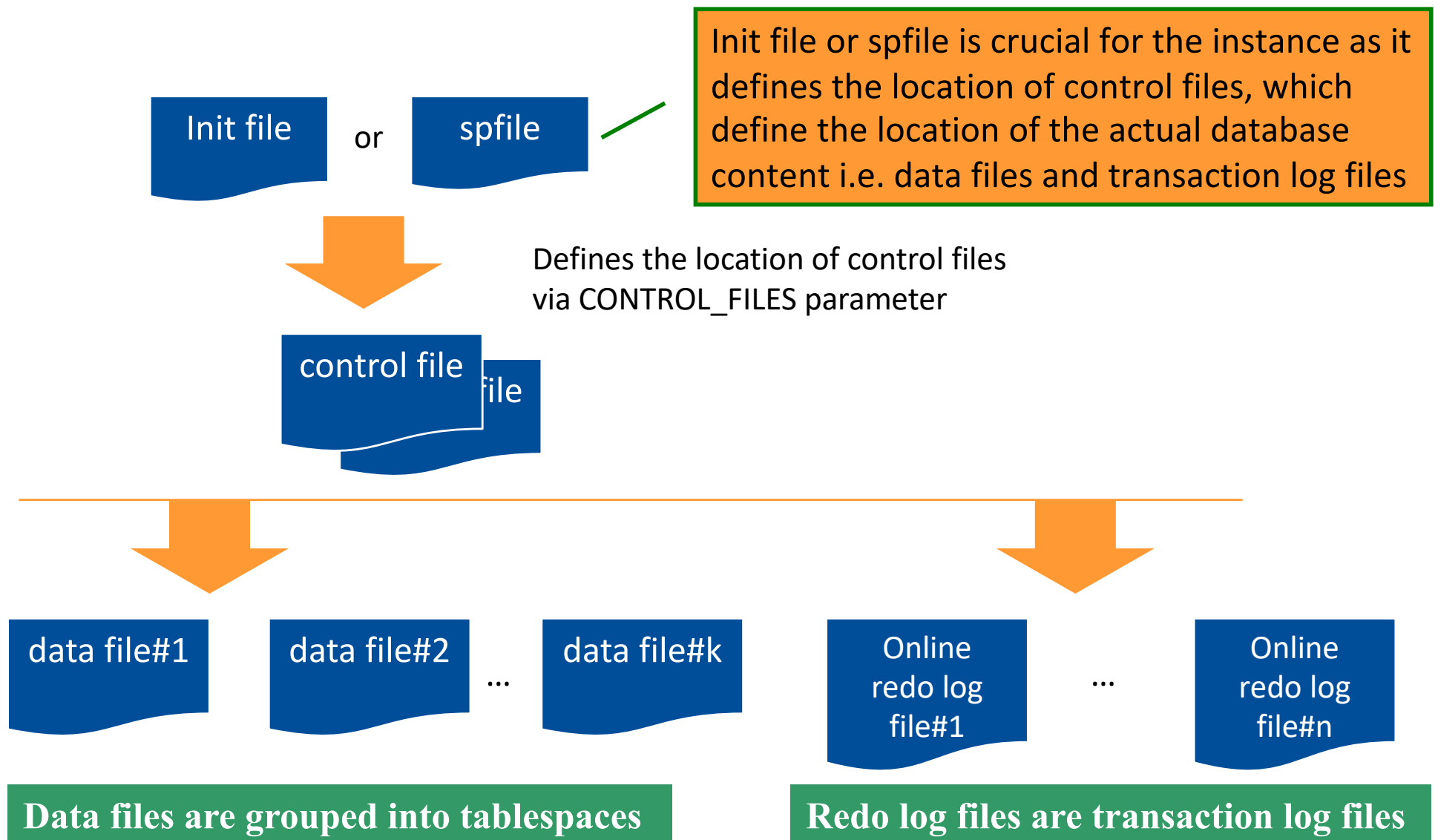
Files configuring the parameters of Oracle instance

Parameter files

- Starting from Oracle 9i R1 two categories of parameter files exist:
 - Initialisation file (init file) i.e. init<ORACLE_SID>.ora i.e. the old solution
 - server parameter file (spfile) i.e. spfile<ORACLE_SID>.ora i.e. the new option
- Both files contain parameter values i.e. pairs: (parameter_name, parameter_value)
- Both categories of the files contain the settings used at the beginning of instance startup to define the instance settings including the location of other crucial files. The administrator of the server decides which of the two standards of parameter files to use

ORACLE_SID stands for site identifier i.e. an identifier of a database. As many databases may exist for the same ORACLE_HOME directory, ORACLE_SID needs to be used to differentiate between them.

File categories – an overview



Other file categories

Control files

Pliki sterujące (kontrolne)

- Small binary files (<64MB) that:
 - describe the structure of database i.e. tell the instance where the database files and redo log files are
 - tell the Oracle database about checkpoints and an archive redo log history
- Required to mount or open database
- Each database has one unique control file kept in many copies
- A database must have a minimum of two files, three are preferred on different disks
- Should one control file be lost, it can be recovered by copying one of the remaining files
- Should be multiplexed either by hardware i.e. using RAID or by explicitly placing them on different disks
- These files are of interest to DBAs only

Redo log files

- Transactional logs for the database
- Used for:
 - Instance recovery,
 - Media recovery after a data file restore from a backup,
 - Standby databases and input into Streams i.e. replication
- There are two categories of redo log files: *online* and *archive* redo log files

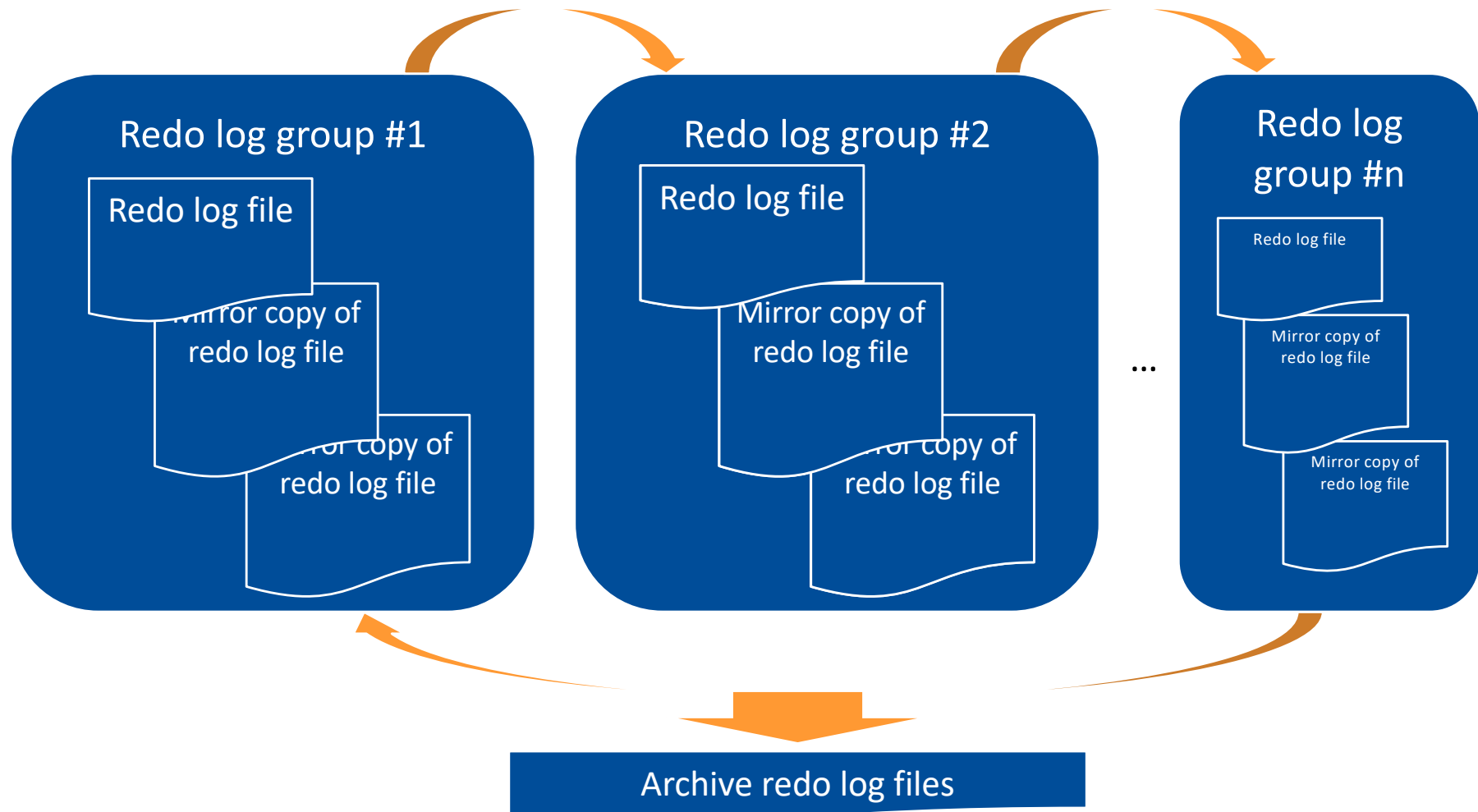
1. Using archive and on-line redo log files the database can be restored to the point in time immediately before the instance failure or to requested time.
2. In the latter case, the database can be restored to a point in time before any DBA's mistake like removing a critical table.

Online redo log files

- There are at least two groups of redo log files. When one of them is active i.e. transactions are being reflected in it, the remaining groups are inactive and can be archived
- Each group consists of one or more redo log members i.e. mirror images of each other
- Redo log files
 - have a fixed size,
 - are used in a circular fashion e.g. in case there are 3 groups, the first group is used, then the second one, the third one and the first one again
- The so called *log switch* occurs when the group is changed. It causes a checkpoint.
- Every time a transaction is committed the information on it is saved into redo log file
- It may happen that the actual changes have not been saved into data files, still the redo log file can be used to restore the changes and reapply them to the database

DBA should ensure that enough online redo log files are allocated so that not to attempt to reuse redo log files before the checkpoint completes. LGWR is the process writing log buffers to online redo log files.

Redo logs and archive logs



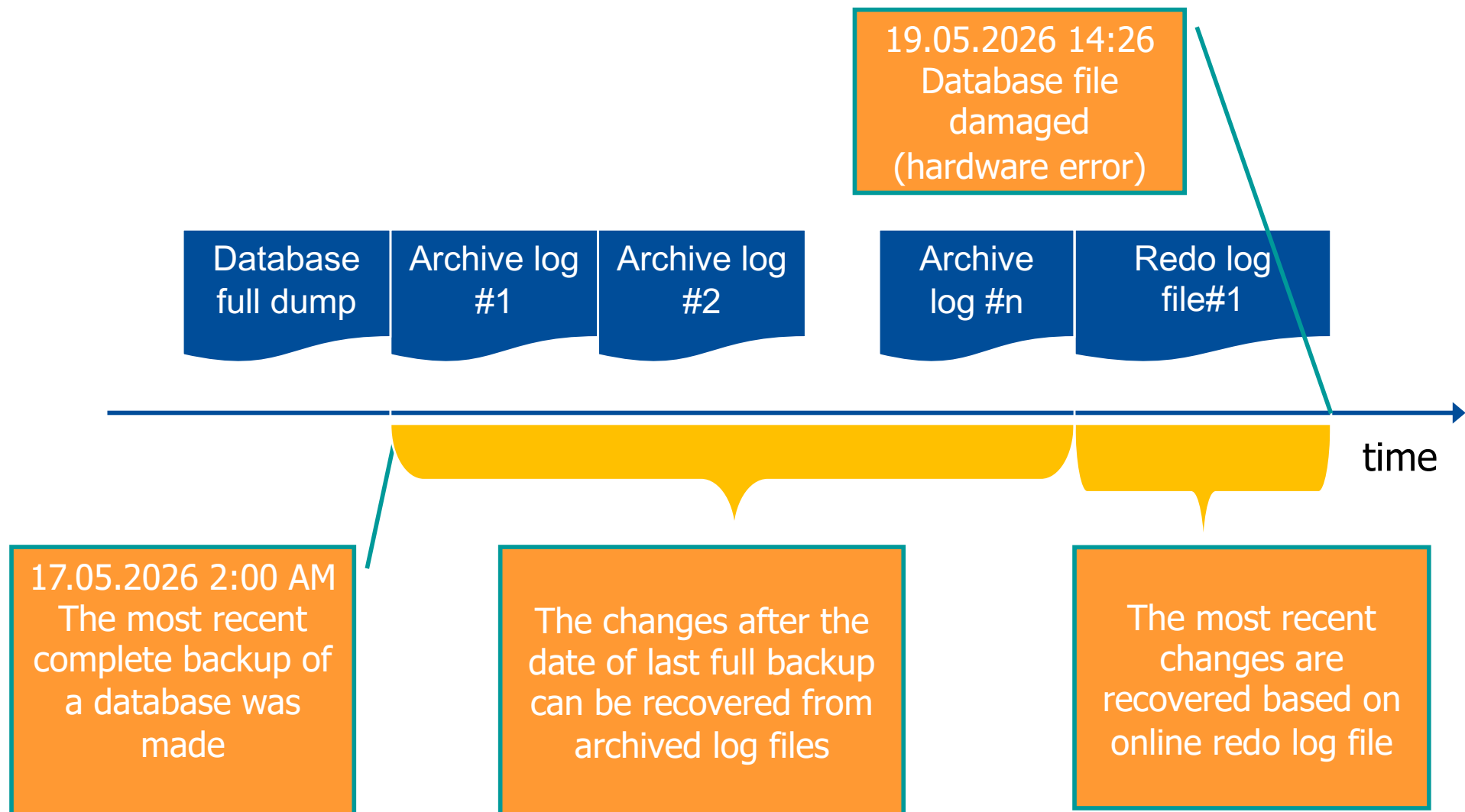
A copy of inactive redo logs into archive logs is periodically made. Such a copy is made from the group not being active at the moment. This is done by Oracle automatically DBMS it is configured to do so.

ARCHIVELOG vs. NOARCHIVELOG

- The Oracle database can run in one of the two modes
- The decision to be made is whether to make a copy of redo log file before reusing it (ARCHIVELOG mode) or not (NOARCHIVELOG)
- Basing on a database backup, a number of archived redo log files and online redo log files, the content of a database can be restored including the last successfully committed transaction
- With NOARCHIVELOG, the database can be restored only:
 - In case the instance is just terminated, until the most recent content of a database, including all successfully committed transactions before instance termination
 - **In case media failure happens, until the point in time of preparing a backup**
 - **In case DBA made an unrecoverable mistake (e.g. table dropped), until the point in time of preparing a backup**
- Thus, any production database should run in ARCHIVELOG mode

Once a database is configured to run in ARCHIVELOG mode, new background process (or many processes) i.e. ARCn (ARC0,ARC1,...) process(es) is/are started.

Database recovery after media failure – sample scenario



Managing backups

Introduction to backups

- Data loss and downtime should be minimised
- Still, due to hardware errors, human mistakes, malfunctioning applications and cybersecurity issues files can get corrupt and/or data can be lost
- Hence, regular backups of data and metadata have to be performed
- Key factors to consider:
 - SLA requirements,
 - Loss of data (acceptable or not),
 - MTBF (mean time between failures)
 - MTTR (mean time to recovery)
 - Budget constraints

Depending on business needs, these factors may play a different role. In some installations it might be crucial to avoid data loss, while in other cases the cost of downtime is an issue of greatest concern.

Backup tools

The following tools can be used to create a copy of data and instance files:

- Operating system utilities capable of copying files. This provides *user-managed backups*.
- Data Pump i.e. a server-based data extraction and import utility, which can be used to:
 - Perform logical backup by querying the data from the tables
 - Restore the content to the point in time when it was created
- **Recovery Manager (RMAN)**, which is the **suggested option** as it can:
 - Be integrated with all the popular tape control systems,
 - Back up data files, control files, archive logs and spfiles,
 - Provides scripting language and public API,
 - Be accessed by using GUI and command line interface.

This provides *server-managed backups* and the most complete functionality.

For an overview of backup tools and their capabilities see

<https://docs.oracle.com/en/database/oracle/oracle-database/19/bradv/introduction-backup-recovery.html>

The need for server-managed backups

- Note that irrespective of DBMS type, the use of just file copies (user-managed backups) or exporting the output of SQL SELECT statements to files is not sufficient
- This is because:
 - DBMS can be operated 24x7. In such cases, there is no time for copying data files as they constantly change
 - Table content can be too large. In such cases, incremental backups containing only the data that has changed since the previous backup are needed
- Therefore, in this lecture (and real system administration) we need to focus on server-managed backups i.e. backups performed with DBMS tools such as RMAN

Server-managed backups – key decisions

- Before making such a backup, some decisions should be made:
 - Should it be closed or open?
 - Should it be whole or partial?
 - Should it be full or incremental?
- In other words, modern DBMS platforms offer a number of ways of performing backups of database content
- This is because creating one large backup of an entire database would be:
 - Taking too much time
 - Slowing down DBMS for too long
 - Increasing recovery time after failure
- The impact (and meaning) of different backup categories is discussed on the next slides

Backup categories – whole vs. partial

- A *whole* (całościowy) backup contains:
 - All the data files for permanent tablespaces,
 - One copy of control file,
 - spfile.
- *Partial* (częściowy) backup:
 - Does not contain all the files found in the whole backup,
 - Takes less time and disk space to complete,
 - Contains selected data files and/or control file,
 - Has to be synchronised with the rest of the database. In such cases, usually archive logs are needed. In other words, to use partial backups to recover the data most likely you need to run the database in archivelog mode.

Online redo log files are not copied. They are protected by multiplexing and archiving. Data files for temporary tablespaces are not copied.

Full or incremental backups?

- *Full (pełny)* backup:
 - Can be whole or partial, but it always contains complete data files
- *Incremental (przyrostowy)* backup:
 - Backup of selected data blocks from data file(s),
 - Can be created with RMAN only. The RMAN determines the list of blocks that have changed since the last full backup.
 - Incremental backups are usually significantly smaller than full backups. On the other hand, to restore a database may mean to use a full backup and a number of incremental backups following it. This is because incremental backups can be *differential* i.e. include only database blocks that have changed since the previous incremental backup.

To create an incremental backup you can not rely on the operating system commands. Simply, the operating system does not know which data blocks have changed.

Closed (offline) or open (online) backups?

- *Offline* a.k.a *closed (zamknięty)* backup:
 - A backup taken when a database is closed i.e. shut down,
 - In other words no users are accessing the database i.e. no data modifications are possible during a period an offline backup is being performed
 - Not very realistic on many production systems
 - a.k.a. *cold* or *consistent*.
- *Online* a.k.a *open (otwarty)* backup:
 - Can only be done when the database runs in archive log mode,
 - When created with operating system commands can get corrupted, as the changes are likely to be applied to data blocks during copying a file.

Real-life strategies

- Frequently rely on a combination of:
 - Closed, whole, full backups made during maintenance slots i.e. on weekly or monthly basis
 - Open, whole, incremental backups performed on daily basis
- Backups are performed during low-load of DBMS i.e. typically at night. This is because of the fact that they usually have a negative impact on the performance of DBMS observed by the users

Backup strategies

- Recommended option:
 - One-time whole database backup,
 - Incremental level 1 backups performed each following day,
 - Automates backup management incl. the recurring backup of changes in the database
- Custom strategies:
 - Individual tablespaces/data files/archived log files can be backed up separately,
 - Oracle backups residing on a disk can be transferred to a tape.

Custom strategies can be more suitable in case numerous applications use different schemas of a database and require individual backup settings.

Summary

- Modern DBMS platforms rely on multiple files to store:
 - initialisation parameters
 - the actual data of relational database(s)
 - temporary data such as the data used for sorting purposes
 - transaction logs
- These files have to be secured to reduce the risk of data loss due to:
 - Hardware failures
 - Human errors
 - Malfunctioning applications
 - Intruders modifying the data
 -
- Hence, data protection includes backups
- Setting up backup strategy is a complex process, as different backup types may be necessary

Appendix I

Key commands and configuration settings

New tablespace: the case of creating a permanent tablespace

```
create tablespace test_space  
datafile 'd:/oradata/test_file.dbf' SIZE 1M;
```

1. The example illustrates the minimal set of options needed to create a new permanent tablespace. Notice that a tablespace to be created must contain at least one datafile.
2. A special category of tablespaces are BIGFILE tablespaces. Such tablespaces can contain only one data file of up to 128TB of data.
3. Remaining options can be set as well.
4. Another method for creating tablespaces is to use Enterprise Manager.
5. **ALTER DATABASE DEFAULT TABLESPACE "test_space"** can be used to set the tablespace to be a default one for newly created users.
6. Tablespace can be altered and dropped.

Control files - multiplexed

```
shutdown immediate;  
host copy  
    D:\oracle10g4\product\10.2.0\oradata\orcl\control01.ctl  
    D:\oracle10g4\product\10.2.0\oradata\orcl\control04.ctl  
startup nomount;  
alter system set  
    control_files="D:\oracle10g4\product\10.2.0\oradata\orcl  
    \control01.ctl", "D:\oracle10g4\product\10.2.0\oradata\or  
    cl\control04.ctl" SCOPE=SPFILE;  
startup force;
```

A new control file is created. Notice that to make it used by an instance, the instance has to be restarted. control_files parameter can be changed with SCOPE=SPFILE only.

The code can be executed in SQLPLUS.

Redo log files - multiplexed

- `ALTER DATABASE ADD LOGFILE MEMBER`
`'D:\ORACLE10G4\PRODUCT\10.2.0\ORADATA\ORCL\myredofile'` TO
GROUP 1 **can be used to add a file to existing group file**
- `alter system switch logfile` should be used then to switch redo groups to make sure that new files can be used successfully
- By default there are three groups of redo log files. At least one extra redo log file should be added to each of the groups to minimise the risk of losing control files.
- Redo log files are crucial for the instance recovery. In case, none of the redo log files in a group can be used, the instance can not be fully recovered.

Archive logs

- Archive logs should be set on. This is done in the following way:
 - `SHUTDOWN IMMEDIATE;`
 - `STARTUP NOMOUNT;`
 - `ALTER DATABASE ARCHIVELOG;`
 - `ALTER DATABASE OPEN;`
 - Once archive logs are turned on, the ARCn background process will make sure redo log files are archived before being overwritten by LGWR process
 - Archive logging is not necessary in most development and testing environments
- Archive log files should be multiplexed

The sequence of commands should be executed in sqlplus. Please note that by default redo logs may be not archived. This means recovery after disk failure may be not possible until the most recent point in time, but only until the time of the most recent backup.

Media recovery – part II

Problem	Solution	Remarks
A non-critical data file lost in ARCHIVE LOG mode	• the database remains available for the users apart from the data stored in the lost file(s) • restore only the missing data file(s) • the files are automatically recovered to current time – using archived logs	Non-critical means not in SYSTEM and UNDO tablespaces
A critical data file lost in ARCHIVE LOG mode	• SHUTDOWN ABORT • STARTUP MOUNT • restore only the missing data file(s) • the files are automatically recovered to current time – using archived logs • ALTER DATABASE OPEN	The only difference comparing to the previous option is the fact that the database has to be shut down and then re-opened

All the production databases should run in ARCHIVELOG. Should it be needed to restore a data file, no data will be lost. The users will not have to re-enter any data. The solutions may differ on different Oracle versions.

Preparing a database for recovery

Database recovery

- To open a database and make it available for the users, the following key files must be present and synchronised:
 - Control files,
 - Online data files,
 - Redo log files (at least one member of each group)
- Next slides illustrates, what a DBA has to practice to be prepared to open a database successfully, by using a combination of STARTUP commands and restoring necessary files from backups to perform recovery process

Media recovery – part I

Problem	Solution	Remarks
A control file lost	• SHUTDOWN ABORT • Recreate a control file by copying another control file • STARTUP	If necessary change the location of a control file by changing the init parameter
A member of redo log file group lost	• The instance is still functioning • Alert log should be checked to find the name(s) of missing files • Restore the missing file by copying an existing member or clear the log group in order to recreate the file(s)	
A data file lost when in NOARCHIVELOG	• SHUTDOWN ABORT • restore the entire database from a backup • STARTUP • Make the users re-enter all the changes since the last backup	All data files and control files have to be restored

Configuring a database for recoverability

Guidelines:

1. Set Mean Time to Recovery (MTTR) target
2. The control files have to be multiplexed i.e. maintained in more than one copy
3. The online redo log files must be multiplexed i.e. there must be more than one member in each group
4. The database must run in ARCHIVELOG mode
5. The archive logs must be multiplexed i.e. there must be more than one member in each group
6. ...backups must be made regularly

**Ideally, copies of multiplexed file should be placed on a different physical device.
Hence, many actions are needed to make data more secure.**

MTTR setting

- `ALTER SYSTEM SET fast_start_mttr_target = 600`
`SCOPE=...` can be used to set MTTR target to a specified time (here 600 seconds i.e. 10 minutes)
- By setting `fast_start_mttr_target`, DBA can specify the frequency of checkpoints so that to avoid SMON processing redo log files for too long during instance recovery. In this case, the instance should be recovered within 10 minutes.
- If the `fast_start_mttr_target` value is too low, the performance of the instance can be largely degraded – simply a lot of instance time will be spent on writing out dirty blocks to the disk.
- A default setting is zero, which maximises performance, but may result in long instance recovery times.

The DBA should take into account the requirements regarding performance and MTTR when deciding about `fast_start_mttr_target`. On some systems MTTR can be accepted to be larger not to sacrifice performance, which turns out to be key issue.

ESTIMATED MTTR parameter, observable in V\$INSTANCE RECOVERY can be investigated to see if instance configuration matches Service Level Agreement (SLA) obligations.

Archive log files

- After switching to ARCHIVELOG, the entire database should be backed up so that archive logs can be actually used to recover a database instance
- Archive log files should be copied to off-line storage and removed regularly. This is to prevent your disk(s) from filling up with a number of archive log files.
- In advanced configurations, destination can be Oracle Net alias i.e. the redo stream can be transferred over the network to a remote database to be applied there, which provides basis for Data Guard.

Backup and recovery – issues to consider

- Data loss and downtime should be minimised
- Oracle can be configured not to lose a single committed record
- Key factors to consider:
 - SLA (Service Level Agreement) requirements,
 - Loss of data (acceptable or not),
 - MTBF (mean time between failures)
 - MTTR (mean time to recovery)
 - Budget constraints

Depending on business needs, key factors may play a different role. In some installations it might be crucial to avoid data loss, while in other cases the cost of downtime is an issue of greatest concern.

Database recovery

Database recovery

- A process automatically started during a change to OPEN state, when at least one data file is not synchronised with a control file,
- The instance uses redo log files and possibly archive redo log files to bring the content of data file(s) up to date i.e. to apply recent data changes to an out-of-data data file
- In case, media recovery is required to access required transaction logs, an error is reported and the instance remains in MOUNT state



**European
Funds**

Knowledge Education Development

**Warsaw University
of Technology**

European Union
European Social Fund



MSc program in Data Science has been developed
as a part of task 10 of the project
„NERW PW. Science - Education - Development - Cooperation”
co-funded by European Union from European Social Fund.